Authentication and Authorization

The foundation of any secure system relies on two distinct but closely connected concepts

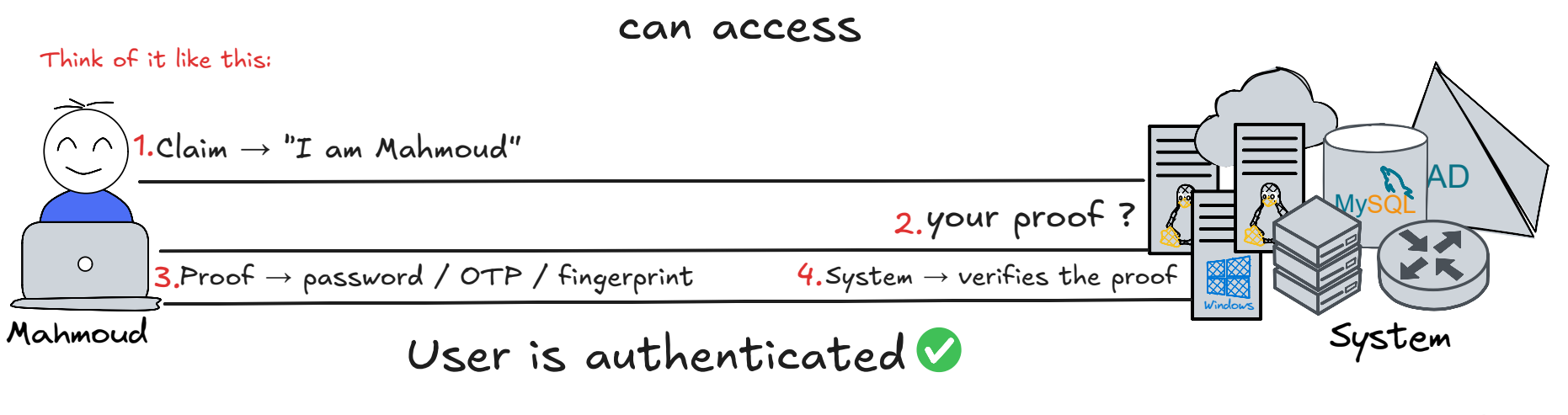
Authentication → Who are You ?

Authorization → What Can You Do ?

Authentication

Authentication verifies who you are, not what you can access

Think of it like this:



1.Claim → "I am Mahmoud"
2.your proof ?
3.Proof → password / OTP / fingerprint
4.System → verifies the proof
Mahmoud
AD
MySQL
Windows
System

User is authenticated ✓

Authentication Factors

Something You Know
- Password
- PIN

Something You Have
- Phone (OTP)
- Hardware token

Something You Are
- Fingerprint
- Face recognition

Combine → Multi-Factor Authentication (MFA) → 3FA / 2FA

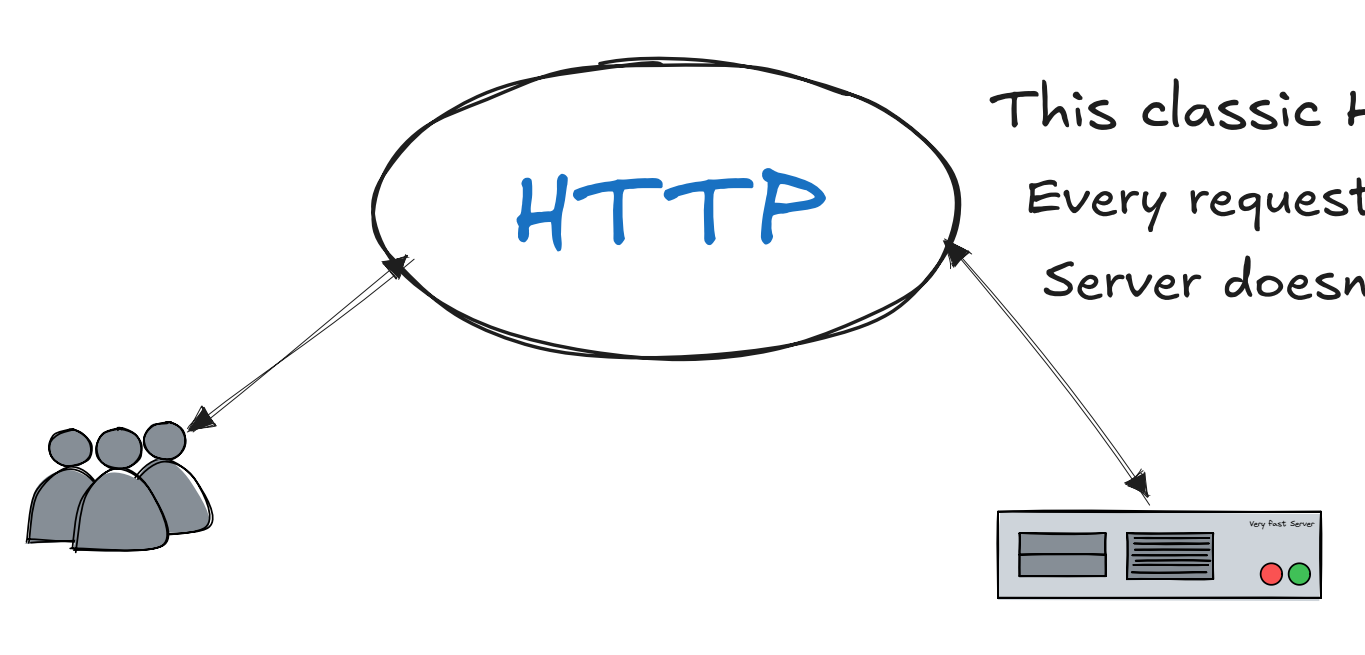
Docker

As technology evolves, stronger authentication methods are required

cloud

---

1. Sessions



This classic HTTP is Statless

Every request = brand new request

Server doesn't remember anything

Early web apps had to re-authenticate every request

Totally inefficient for complex apps

The Solution: Stateful Sessions

A way for the server to remember a user between multiple requests

How Session Work?

1. User logs in
2. Server creates a session record
   - session_id = random token
   - Stores in memory or database
   - Associates with user info (userID, roles, etc.)
3. Server sends session ID to client
   - Usually in a cookie
   - `Cookie: Set-Cookie: session_id=abc123; HttpOnly; Secure`
4. Client includes cookie in future requests
   - `GET /dashboard`
   - `Cookie: session_id=abc123`
5. Server validates session ID
   - Checks memory / DB
   - Maps to user → grants access

Why This Works

Server now remembers state

You don't need to send credentials every request

Enables:
- Role-based access
- Persistent login
- Shopping carts, personalization

⚠ Tradeoffs:

Stateful sessions need server storage → more memory / DB lookups

Harder to scale in distributed systems

Requires secure session management

Stateful Became a Bottleneck ⚠

The Need for a balance statelessness and statefulness:

2. JWT (JSON Web Token)

stateless, self-contained token

Encoded

Base-64

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMj
M0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiYWRtaW4iOnRyd
WUsImlhdCI6MTUxNjIzOTAyMn0.KMUFsIDTnFmyG3nMiGM6H9FN
FUR0f3wh7SmqJp-QV30
```

Decoded

The Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

MetaData: هنا بيعرف السرفر نوع التوكن و خوارزميه التشفير

The Payload

JSON Object

```
{
"sub": "1234567890",
  "name": "Kage",
  "role": "admin",
 "iat": 1516239022
}
```

Claim: الجزء اللي بيشيل الـ Claims (الداتا اللي إنت عايز تنقلها زي الـ User ID والـ Role).

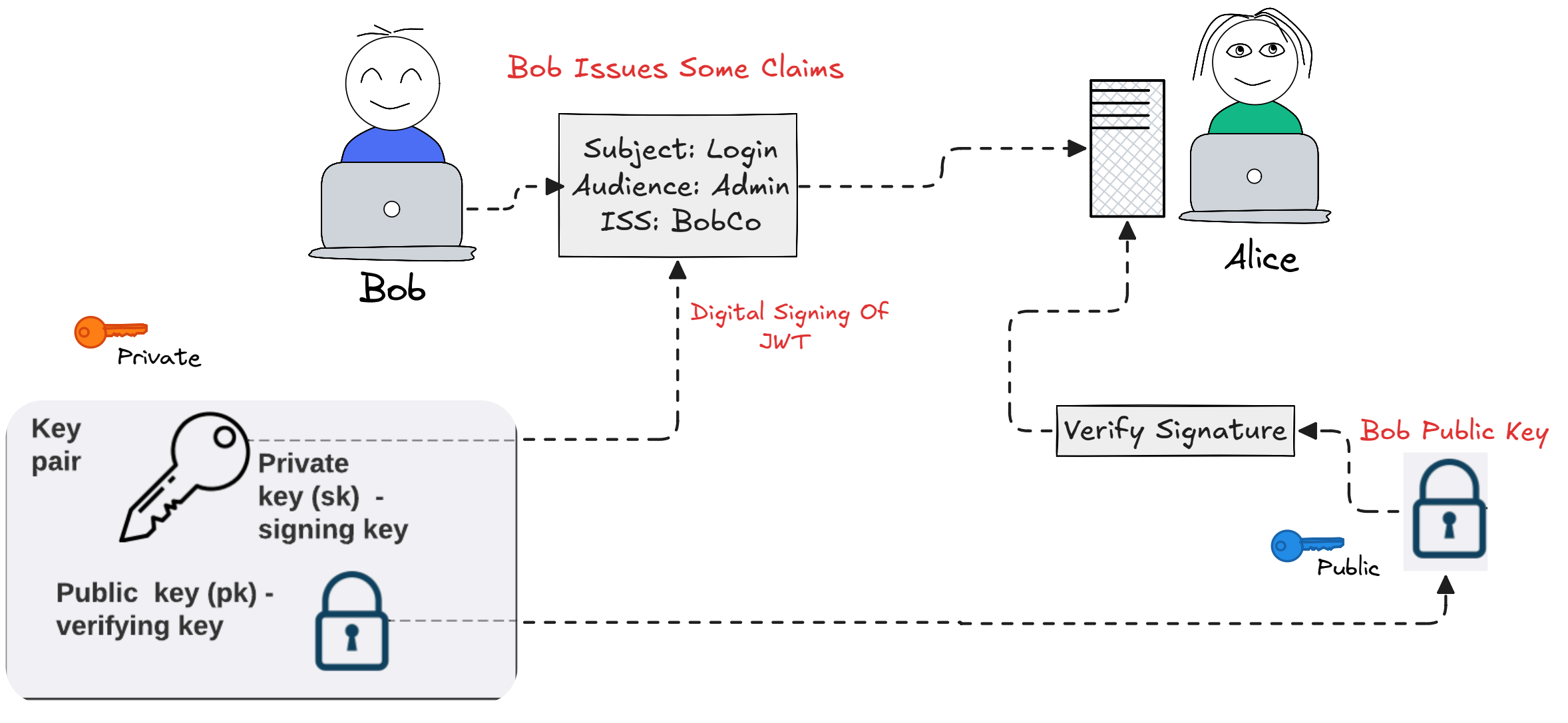
The Signature

```
HMACSHA256(
base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your_256_bit_secret_key
)
```

→ Secret Key

Security Stamp: اللي بيضمن إن محدش لعب في الـ Header أو الـ Payload.

Asymmetric Cryptography

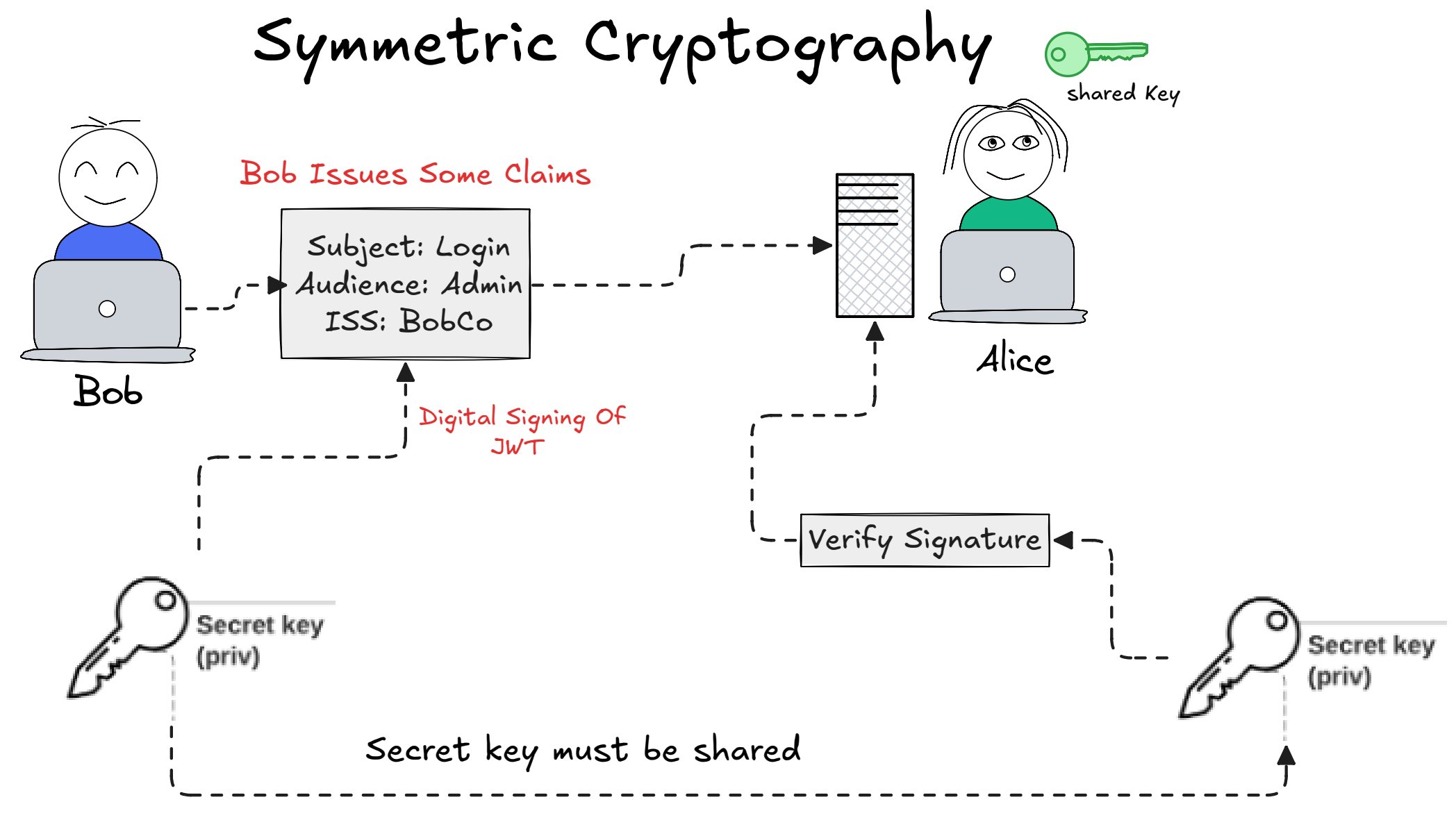


Bob Issues Some Claims
Subject: Login
Audience: Admin
ISS: BobCo
Bob
Alice
Digital Signing Of JWT
Private
Key pair
Private key (sk) - signing key
Public key (pk) - verifying key
Verify Signature
Bob Public Key
Public

"Alice can use the Public Key to verify the signature, ensuring that the token was authentically issued by Bob and hasn't been tampered with. However, she can never generate a new token herself because she doesn't possess the Private Key."

Symmetric Cryptography

shared Key



Bob Issues Some Claims
Subject: Login
Audience: Admin
ISS: BobCo
Bob
Alice
Digital Signing Of JWT
Verify Signature
Secret key (priv)
Secret key must be shared

"Both Bob and Alice share the exact same Secret Key. Bob uses it to sign the token, and Alice uses that identical key to verify the signature. However, the major risk here is that if this shared Secret Key is ever compromised, anyone who gets it can forge new, fully valid tokens."

---

3. Cookie

Storage Mechanism

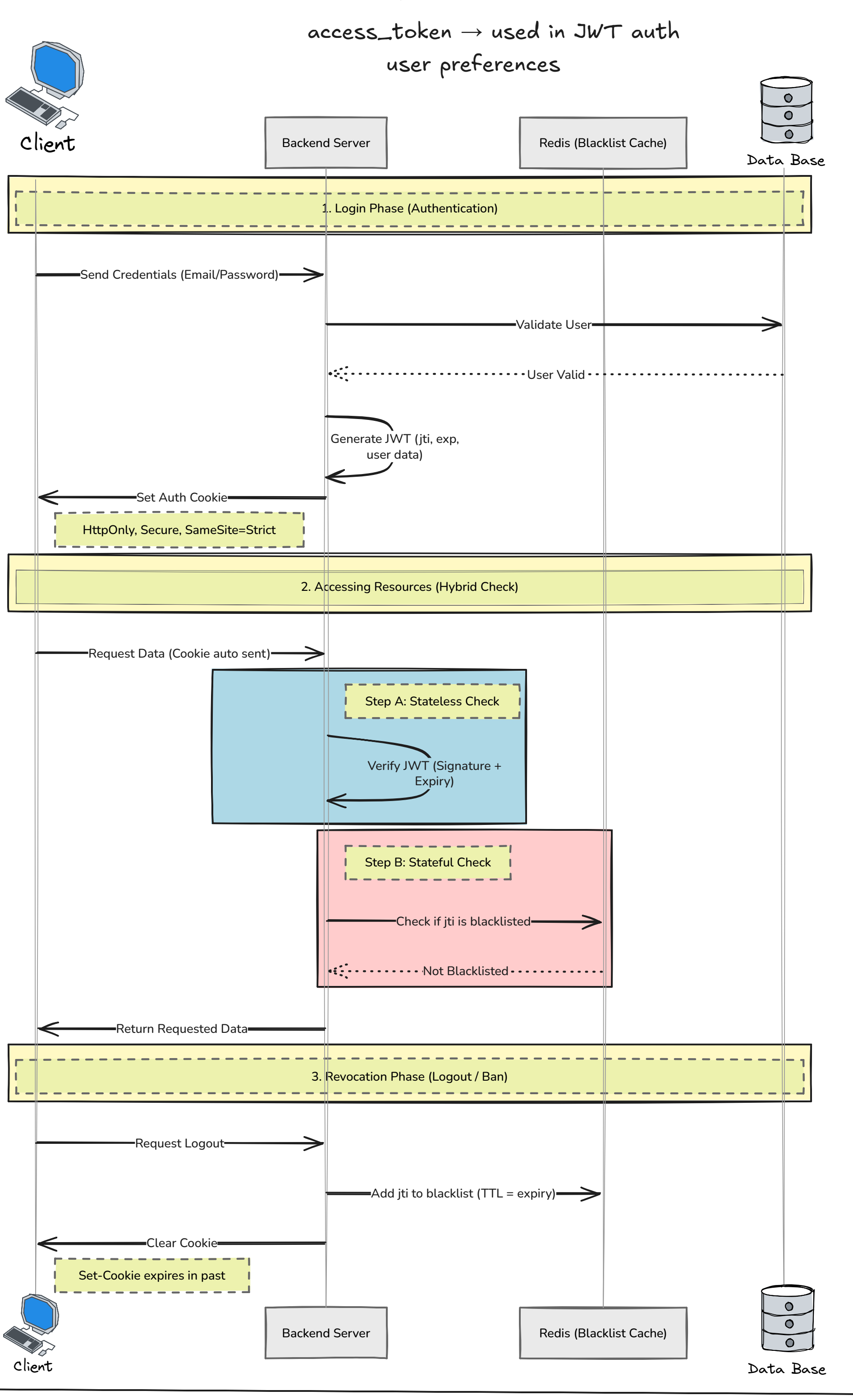
A cookie is a client-side storage mechanism used to persist and automatically send data (like session IDs or tokens) with HTTP requests. A cookie is just a carrier of information

Cookies can store things like:

session_id → used in Sessions `Cookie: sessionid=123456`

access_token → used in JWT auth

user preferences

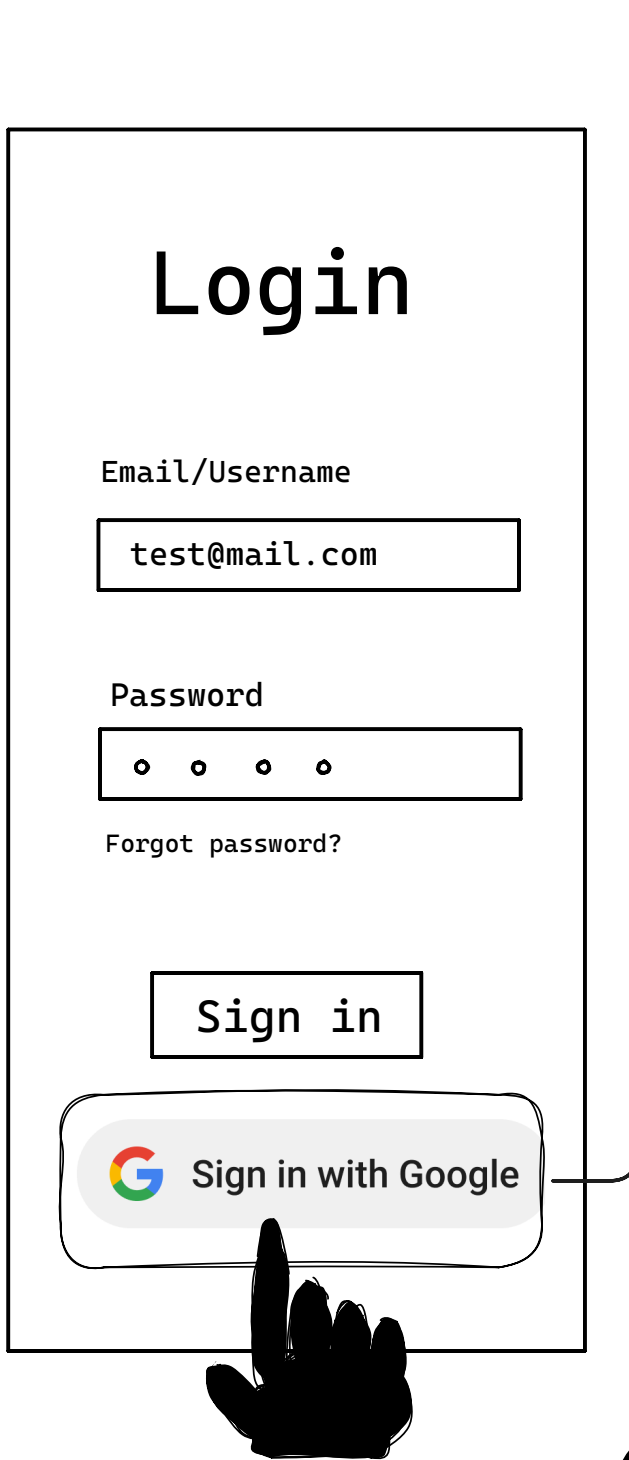


Client
Backend Server
Redis (Blacklist Cache)
Data Base
1. Login Phase (Authentication)
Send Credentials (Email/Password)
Validate User
User Valid
Generate JWT (jti, exp, user data)
Set Auth Cookie
HttpOnly, Secure, SameSite=Strict
2. Accessing Resources (Hybrid Check)
Request Data (Cookie auto sent)
Step A: Stateless Check
Verify JWT (Signature + Expiry)
Step B: Stateful Check
Check if jti is blacklisted
Not Blacklisted
Return Requested Data
3. Revocation Phase (Logout / Ban)
Request Logout
Add jti to blacklist (TTL = expiry)
Clear Cookie
Set-Cookie expires in past
Client
Backend Server
Redis (Blacklist Cache)
Data Base

---

3. OAuth 2.0 (Delegated Authentication)

You trust Google instead of handling password



Login
Email/Username
test@mail.com
Password
Forgot password?
Sign in
Sign in with Google

You're NOT giving your password to the website

You're letting Google say:

"Yes, this user is real"

1. User clicks:
   Login with Google
2. Redirect to Google
   your-app.com → accounts.google.com
3. User logs in on Google
   (Your app never sees the password)
4. Google asks:
   Do you allow this app to access your email?
5. Google sends back:
   Authorization Code
6. Your backend exchanges it for:
   Access Token

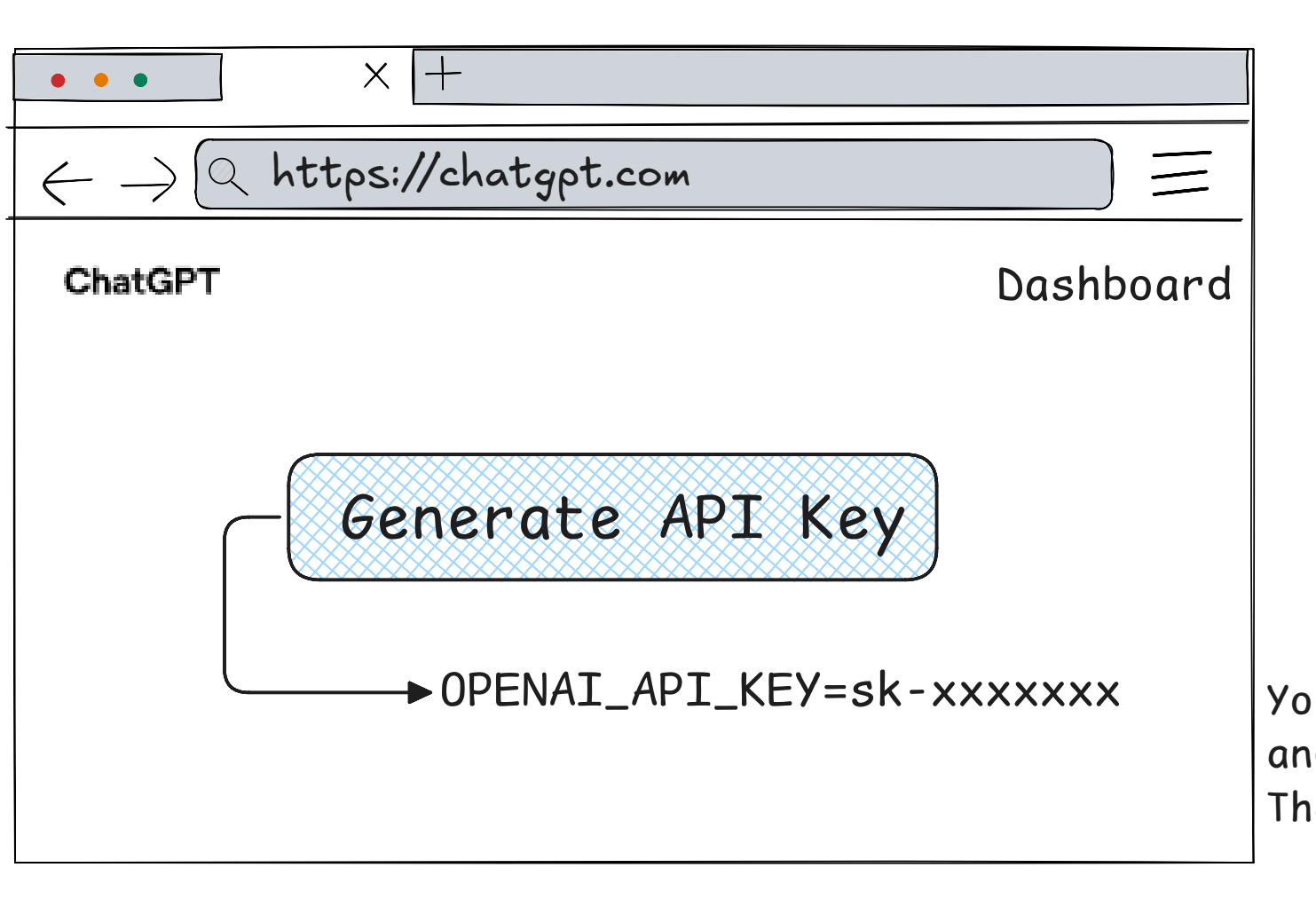
4. API KEY

API Keys = Authentication + Simple Authorization

API Keys are a form of authentication designed specifically for machine-to-machine communication, unlike traditional stateful (sessions) or stateless (JWT) methods used for human users.

An API key is a cryptographically secure random string generated by a platform, used to identify and authenticate a client application making requests to its API.

Use Case: Access GPT Model Without UI



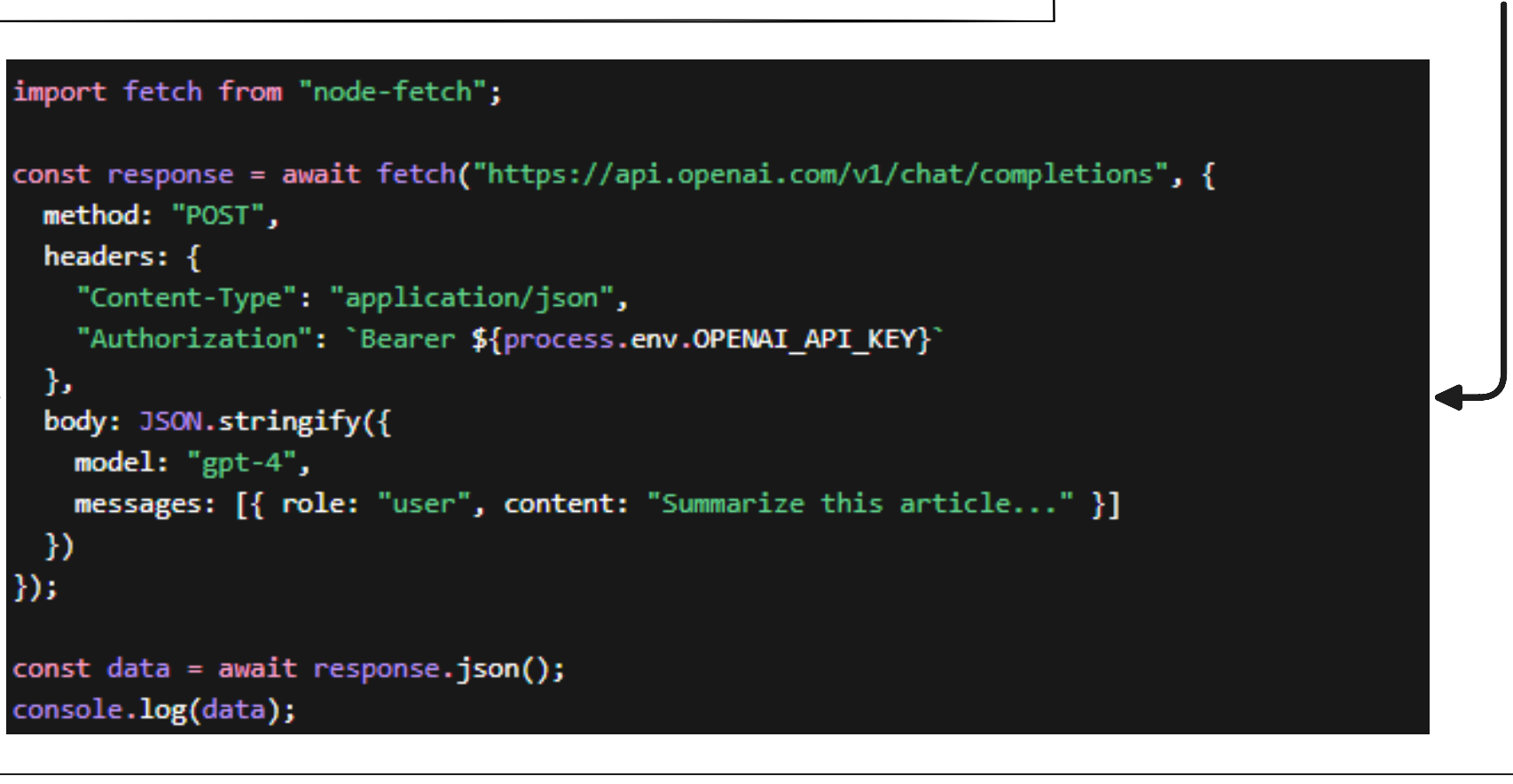
https://chatgpt.com
ChatGPT
Dashboard
Generate API Key

`OPENAI_API_KEY=sk-xxxxxxx`

MY Server

Your server Will Make THE API Request and The API Key Will Let GPT API That This Server is authorized

Request



```
import fetch from "node-fetch";

const response = await fetch("https://api.openai.com/v1/chat/completions", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${process.env.OPENAI_API_KEY}`
  },
  body: JSON.stringify({
    model: "gpt-4",
    messages: [{ role: "user", content: "Summarize this article..." }]
  })
});

const data = await response.json();
console.log(data);
```

GPt Server

Open AI

Future

Decentralized Identity (DID) Blockchain

Your identity = stored on company servers (Google, Facebook, etc.) ⚠

Your identity = owned and controlled by YOU ✓

No central database needed

Behavioral Biometrics

The system tracks patterns like:

- Typing speed
- Typing rhythm
- Mouse movement
- Touch pressure (mobile)

→ Current behavior vs stored behavior

Match → allow ✓

Suspicious → trigger extra verification ❗

Post-Quantum Cryptography

Today's security depends on problems like:

factoring large numbers (RSA)

discrete logarithms (ECC)

→ Quantum computers (future) can break this easily

We design new algorithms that: → Even quantum computers can't break

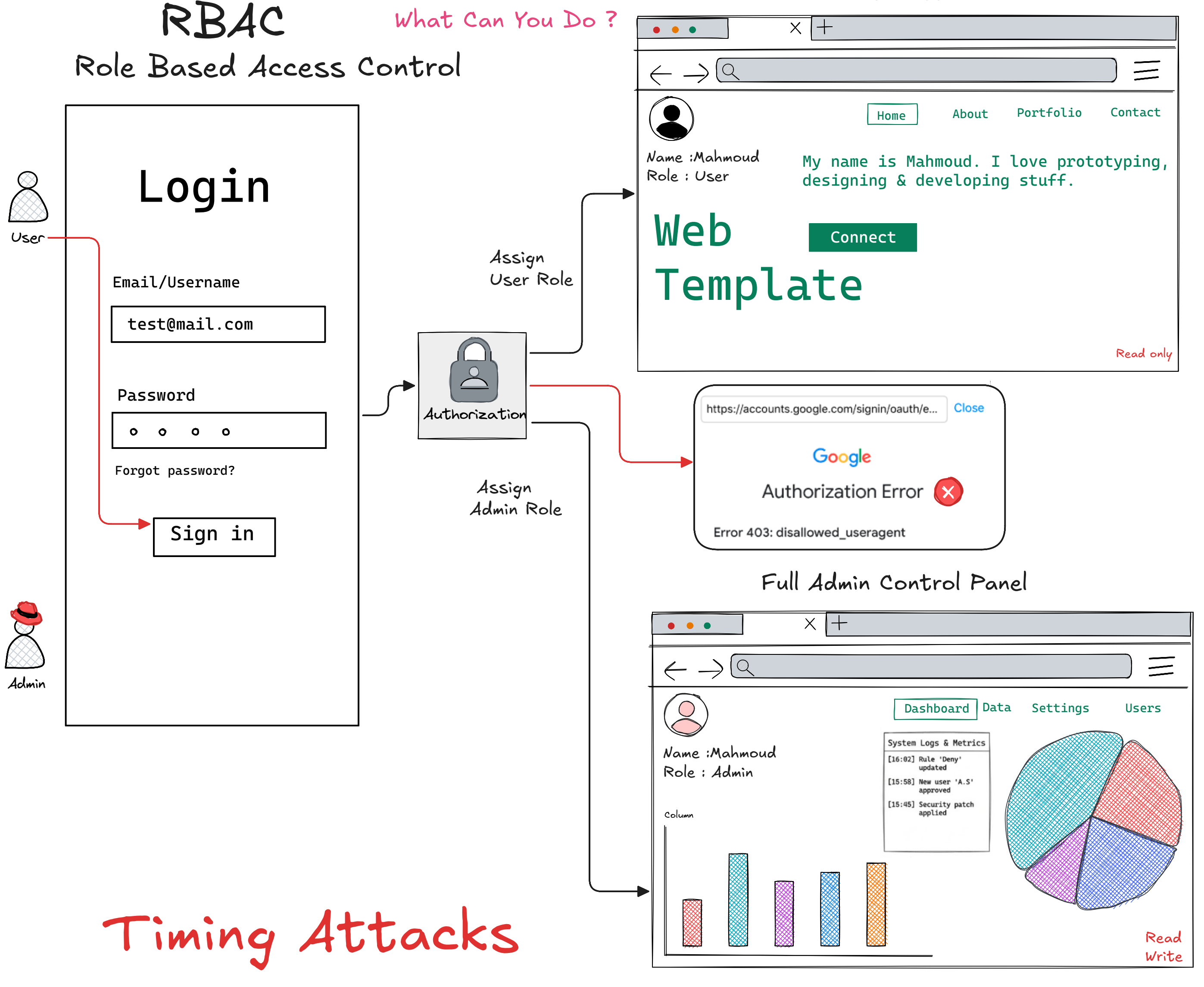
---

Authorization

What Can You Do ?

RBAC

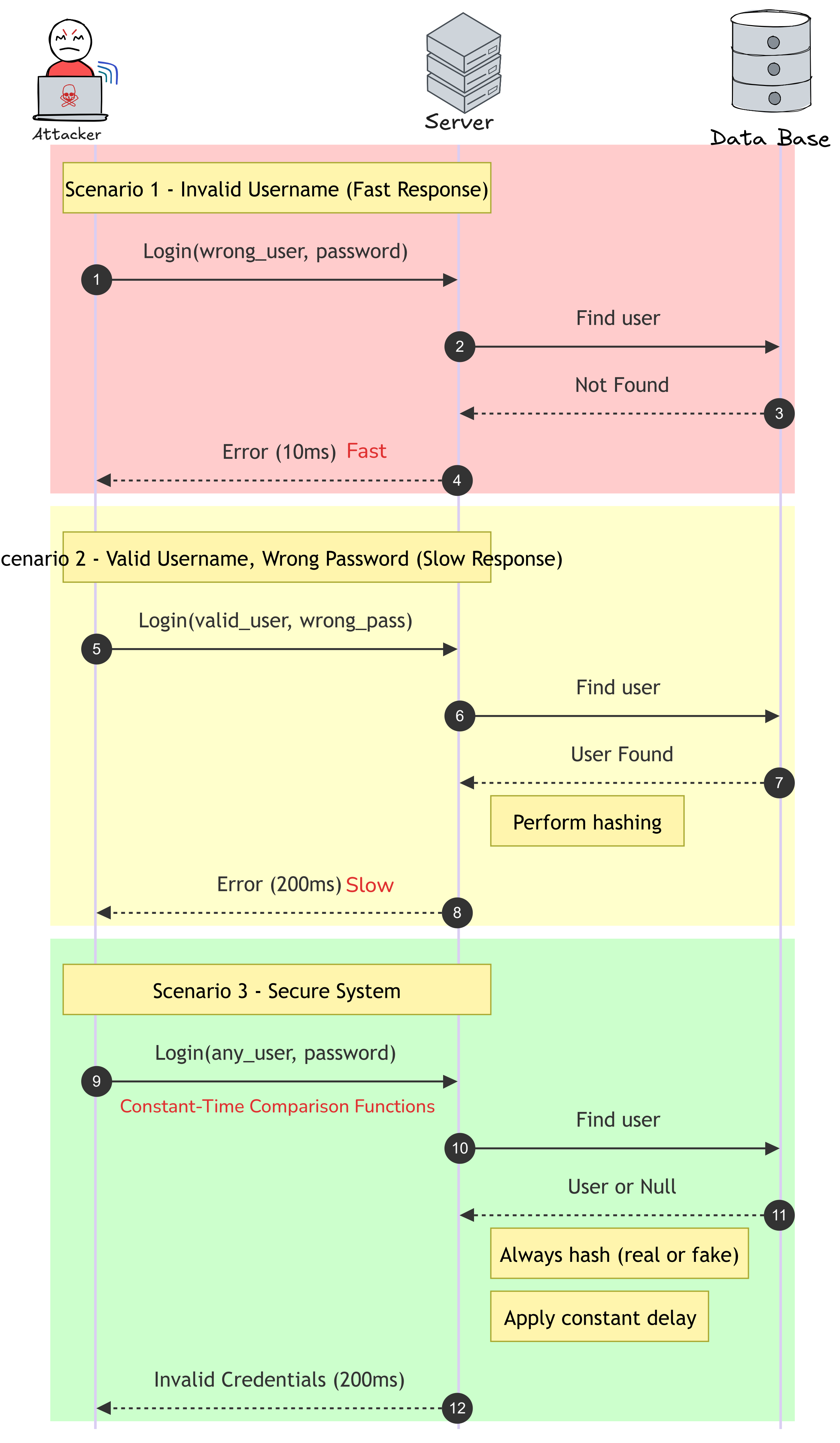
Role Based Access Control



User
Admin
Login
Email/Username
test@mail.com
Password
Forgot password?
Sign in
Authorization
Assign User Role
Assign Admin Role
Limited User Portal
Home
About
Portfolio
Contact
Name :Mahmoud
Role : User
My name is Mahmoud. I love prototyping, designing & developing stuff.
Web Template
Connect
Read only
https://accounts.google.com/signin/oauth/e...
Close
Google
Authorization Error
Error 403: disallowed_useragent
Full Admin Control Panel
Dashboard
Data
Settings
Users
Name :Mahmoud
Role : Admin
Column
System Logs & Metrics
Read Write

Timing Attacks

A Timing Attack is a side-channel attack where the attacker gains information by measuring how long a system takes to respond.



Attacker
Server
Data Base
Scenario 1 - Invalid Username (Fast Response)
1 Login(wrong_user, password)
2 Find user
3 Not Found
4 Error (10ms) Fast
Scenario 2 - Valid Username, Wrong Password (Slow Response)
5 Login(valid_user, wrong_pass)
6 Find user
7 User Found
Perform hashing
8 Error (200ms) Slow
Scenario 3 - Secure System
9 Login(any_user, password)
Constant-Time Comparison Functions
10 Find user
11 User or Null
Always hash (real or fake)
Apply constant delay
12 Invalid Credentials (200ms)

Attacker Measures response time differences to infer sensitive information and then uses brute-force attacks to guess (like whether a user exists) .passwords, all without breaking the encryption itself ❗

Error messages can be dangerous because they may reveal too much information to an attacker, like whether a username exists or what part of the login failed, making attacks like user enumeration and brute-force much easier.

Allways Send a generics Error message Through The Auth Flow ✓

📎 Cross-Links:

📍- Related to: BOM Session Storage, BOM Local Storage